ACADEMIC PRACTICAL REFERENCE & QUESTION BANK

Natural Language Processing

Complete BCA Semester Practical Exam Lab Solutions (Programs 01 to 42)

42	6	100%
PROGRAMS	SYLLABUS MODULES	TESTED & VERIFIED

Key Features of This Lab Manual

- **Concise & Exam-Ready:** Each program is written in 5–20 lines of clear, idiomatic Python 3, making it straightforward to memorize and write during lab exams.
- **Authentic Algorithms:** Implements genuine NLP concepts including NLTK Tokenization, CFG/PCFG Parsers, WordNet similarity, HMM, CRF, TF-IDF, and Naïve Bayes / SVM.
- **Verified Outputs:** Every problem contains verified terminal outputs matching the exact code execution.
- **Modern Stack:** Fully compatible with Python 3.10+ through 3.13, NLTK 3.9+, Scikit-Learn 1.3+, SciPy, NumPy, and NetworkX.

Natural Language Processing (NLP) Lab Record • Complete Programs 01 – 42 • Python 3 Solutions

Table of Contents

Quick index of all 42 syllabus practical programs and problem statements

#	Problem Statement / Topic	Module	Source File
01	Tokenize a paragraph into sentences and words using NLTK	Module 1	01_tokenize_paragraph_nltk.py
02	Build a simple unigram language model for a given corpus	Module 1	02_unigram_language_model.py
03	Implement a rule-based system to identify parts of a sentence	Module 1	03_rule_based_parts_of_sentence.py
04	Perform basic semantic analysis using word similarity	Module 1	04_word_similarity_semantic_analysis.py
05	Demonstrate pragmatics by identifying speech acts	Module 1	05_pragmatics_speech_acts.py
06	Use Python libraries to analyze speech signals	Module 1	06_analyze_speech_signals.py
07	Convert text to phoneme representation using a phonetic dictionary	Module 1	07_text_to_phoneme_cmudict.py
08	Write a program to demonstrate ASCII vs Unicode encoding	Module 1	08_ascii_vs_unicode_encoding.py
09	Handle multilingual text input and normalization in Python	Module 1	09_multilingual_text_normalization.py
10	Implement text preprocessing: tokenization, stopword removal, stemming, lemmatization	Module 1	10_text_preprocessing_pipeline.py
11	Load and explore a corpus using NLTK	Module 2	11_load_and_explore_nltk_corpus.py
12	Access and query WordNet for synonyms, antonyms, and hypernyms	Module 2	12_wordnet_synonyms_antonyms_hypernyms.py
13	Parse and extract information from an XML-based linguistic resource	Module 2	13_parse_xml_linguistic_resource.py
14	Write regex patterns to extract emails, dates, and URLs from text	Module 2	14_regex_extract_emails_dates_urls.py
15	Implement a simple Finite State Automaton (FSA) for word recognition	Module 2	15_finite_state_automaton_word_recognition.py
16	Implement a simple morphological analyzer (prefix, suffix stripping)	Module 2	16_morphological_analyzer_affix_stripping.py

#	Problem Statement / Topic	Module	Source File
17	Implement a simple Hidden Markov Model (HMM) for sequence prediction	Module 3	`17_hidden_markov_model_sequence_prediction.py`
18	Train a classifier using Naïve Bayes / SVM for text classification	Module 3	`18_naive_bayes_svm_text_classification.py`
19	Apply Conditional Random Fields (CRF) for sequence labeling (NER)	Module 3	`19_crf_sequence_labeling_ner.py`
20	Perform POS tagging using NLTK	Module 3	`20_pos_tagging_nltk.py`
21	Implement a stochastic POS tagger using HMM	Module 3	`21_stochastic_pos_tagger_hmm.py`
22	Demonstrate Transformation-Based Learning (TBL) tagging	Module 3	`22_transformation_based_learning_tbl.py`
23	Handle unknown words in POS tagging	Module 3	`23_handle_unknown_words_pos_tagging.py`
24	Identify named entities and multiword expressions	Module 3	`24_named_entities_and_multiword_expressions.py`
25	Define a Context-Free Grammar (CFG) in Python and parse sentences	Module 4	`25_cfg_define_and_parse_sentences.py`
26	Implement top-down and bottom-up parsing	Module 4	`26_top_down_and_bottom_up_parsing.py`
27	Use chart parsing to analyze ambiguous sentences	Module 4	`27_chart_parsing_ambiguous_sentences.py`
28	Implement probabilistic parsing using PCFG	Module 4	`28_probabilistic_parsing_pcfg.py`
29	Load and analyze TreeBank data using NLTK	Module 4	`29_load_and_analyze_treebank.py`
30	Evaluate parsing performance using precision and recall	Module 4	`30_evaluate_parsing_precision_recall.py`
31	Visualize parse trees for given sentences	Module 4	`31_visualize_parse_trees.py`
32	Represent sentence meaning using logical forms	Module 5	`32_sentence_meaning_logical_forms.py`
33	Perform lexical semantic analysis using WordNet similarity measures	Module 5	`33_lexical_semantics_wordnet_similarity.py`
34	Build a semantic network from a small dataset	Module 5	`34_semantic_network_dataset.py`

#	Problem Statement / Topic	Module	Source File
35	Implement the Lesk algorithm for WSD	Module 5	`35_lesk_algorithm_wsd.py`
36	Use WordNet to disambiguate word meanings in context	Module 5	`36_wordnet_disambiguate_meanings_in_context.py`
37	Build a supervised WSD model using machine learning	Module 5	`37_supervised_wsd_machine_learning.py`
38	Analyze text coherence using sentence similarity	Module 6	`38_text_coherence_sentence_similarity.py`
39	Build a simple Machine Translation system using sequence models	Module 6	`39_simple_machine_translation_sequence_model.py`
40	Implement Information Retrieval using TF-IDF	Module 6	`40_information_retrieval_tfidf.py`
41	Perform text classification (spam detection / sentiment analysis)	Module 6	`41_text_classification_spam_sentiment.py`
42	Generate text summaries using extractive methods	Module 6	`42_extractive_text_summarization.py`

MODULE 1

Foundations, Tokenization & Preprocessing

PROGRAM 01 `01_tokenize_paragraph_nltk.py` Foundations, Tokenization & Preprocessing

PROBLEM STATEMENT:

Tokenize a paragraph into sentences and words using NLTK

<> PYTHON 3 SOURCE CODE

```python
import nltk
from nltk.tokenize import sent_tokenize, word_tokenize

text = "Hello world! Welcome to NLP. Let us learn."

print("Sentences:", sent_tokenize(text))
print("Words:    ", word_tokenize(text))
```

>_ EXPECTED TERMINAL OUTPUT

bash terminal • python 01_tokenize_paragraph_nltk.py

```
Sentences: ['Hello world!', 'Welcome to NLP.', 'Let us learn.']
Words:     ['Hello', 'world', '!', 'Welcome', 'to', 'NLP', '.', 'Let', 'us', 'learn', '.']
```

PROGRAM 02 `02_unigram_language_model.py` Foundations, Tokenization & Preprocessing

PROBLEM STATEMENT:

Build a simple unigram language model for a given corpus

<> PYTHON 3 SOURCE CODE

```python
from collections import Counter

corpus = "the cat sat on the mat and the cat purred".split()
total = len(corpus)
counts = Counter(corpus)
unigram_probs = {w: count / total for w, count in counts.items()}

# Calculate probability of a test sentence
sent = "the cat purred".split()
prob = 1.0
for w in sent:
    prob *= unigram_probs.get(w, 0)

print("Sample Word Probabilities:", {k: round(v, 2) for k, v in list(unigram_probs.items())[:3]})
print(f"P('the cat purred'): {prob:.4f}")
```

>_ EXPECTED TERMINAL OUTPUT

bash terminal • python 02_unigram_language_model.py

```
Sample Word Probabilities: {'the': 0.3, 'cat': 0.2, 'sat': 0.1}
P('the cat purred'): 0.0060
```

PROGRAM 03 `03_rule_based_parts_of_sentence.py` Foundations, Tokenization & Preprocessing

PROBLEM STATEMENT:

Implement a rule-based system to identify parts of a sentence

<> PYTHON 3 SOURCE CODE

```python
import nltk

sent = "The student solved the puzzle"
tags = nltk.pos_tag(nltk.word_tokenize(sent))

# Rules: words before verb = Subject, verb = Predicate, words after = Object
subject, verb, obj = [], [], []
state = 'SUBJ'
for w, t in tags:
    if t.startswith('VB'):
        state = 'VERB'
        verb.append(w)
    elif state == 'VERB':
        state = 'OBJ'
        obj.append(w)
    elif state == 'SUBJ':
        subject.append(w)
    else:
        obj.append(w)

print("Subject:  ", " ".join(subject))
print("Predicate:", " ".join(verb))
print("Object:   ", " ".join(obj))
```

>_ EXPECTED TERMINAL OUTPUT

bash terminal • python 03_rule_based_parts_of_sentence.py

```
Subject:   The student
Predicate: solved
Object:    the puzzle
```

PROGRAM 04 `04_word_similarity_semantic_analysis.py` Foundations, Tokenization & Preprocessing

PROBLEM STATEMENT:

Perform basic semantic analysis using word similarity

<> PYTHON 3 SOURCE CODE

```python
from nltk.corpus import wordnet as wn

w1 = wn.synsets("cat", pos=wn.NOUN)[0]
w2 = wn.synsets("dog", pos=wn.NOUN)[0]

print("Path Similarity:     ", w1.path_similarity(w2))
print("Wu-Palmer Similarity:", round(w1.wup_similarity(w2), 4))
```

>_ EXPECTED TERMINAL OUTPUT

bash terminal • python 04_word_similarity_semantic_analysis.py

```
Path Similarity:      0.2
Wu-Palmer Similarity: 0.8571
```

PROGRAM 05 `05_pragmatics_speech_acts.py` Foundations, Tokenization & Preprocessing

PROBLEM STATEMENT:

Demonstrate pragmatics by identifying speech acts

<> PYTHON 3 SOURCE CODE

```python
def identify_speech_act(text):
    if text.endswith('?'): return "Question"
    if text.endswith('!'): return "Exclamation"
    if text.split()[0].lower() in ['please', 'close', 'open', 'stop', 'do']: return "Command"
    return "Statement"

for s in ["What is your name?", "Close the door.", "NLP is amazing!"]:
    print(f"'{s}' -> {identify_speech_act(s)}")
```

>_ EXPECTED TERMINAL OUTPUT

bash terminal • python 05_pragmatics_speech_acts.py

```
'What is your name?' -> Question
'Close the door.' -> Command
'NLP is amazing!' -> Exclamation
```

PROGRAM 06 `06_analyze_speech_signals.py` Foundations, Tokenization & Preprocessing

PROBLEM STATEMENT:

Use Python libraries to analyze speech signals

<> PYTHON 3 SOURCE CODE

```python
import numpy as np
from scipy.io import wavfile

# Generate a synthetic speech-frequency sine tone (440 Hz)
rate = 16000
signal = (np.sin(2 * np.pi * 440 * np.linspace(0, 1, rate)) * 32767).astype(np.int16)
wavfile.write("sample.wav", rate, signal)

sr, data = wavfile.read("sample.wav")
data = data.astype(float)
print(f"Sampling Rate: {sr} Hz, Duration: {len(data)/sr:.1f}s")
print(f"Max Amplitude: {np.max(data)}, RMS Energy: {np.sqrt(np.mean(data**2)):.2f}")
```

>_ EXPECTED TERMINAL OUTPUT

```
bash terminal • python 06_analyze_speech_signals.py

Sampling Rate: 16000 Hz, Duration: 1.0s
Max Amplitude: 32766.0, RMS Energy: 23168.42
```

PROGRAM 07 `07_text_to_phoneme_cmudict.py` Foundations, Tokenization & Preprocessing

PROBLEM STATEMENT:

Convert text to phoneme representation using a phonetic dictionary

<> PYTHON 3 SOURCE CODE

```python
import nltk
from nltk.corpus import cmudict

d = cmudict.dict()
words = "natural language python".split()

for w in words:
    if w in d:
        print(f"{w:<10} -> {' '.join(d[w][0])}")
```

>_ EXPECTED TERMINAL OUTPUT

```
bash terminal • python 07_text_to_phoneme_cmudict.py

natural    -> N AE1 CH ER0 AH0 L
language   -> L AE1 NG G W AH0 JH
python     -> P AY1 TH AA0 N
```

PROGRAM 08 `08_ascii_vs_unicode_encoding.py` Foundations, Tokenization & Preprocessing

PROBLEM STATEMENT:

Write a program to demonstrate ASCII vs Unicode encoding

<> PYTHON 3 SOURCE CODE

```
text = "A ©"

print("ASCII code of 'A':", ord('A'), "| Unicode code of '©':", ord('©'))
print("UTF-8 bytes:      ", text.encode('utf-8'))
print("ASCII with replace:", text.encode('ascii', errors='replace'))
```

>_ EXPECTED TERMINAL OUTPUT

bash terminal • python 08_ascii_vs_unicode_encoding.py

```
ASCII code of 'A': 65 | Unicode code of '©': 169
UTF-8 bytes:       b'A \xc2\xa9'
ASCII with replace: b'A ?'
```

PROGRAM 09 `09_multilingual_text_normalization.py` Foundations, Tokenization & Preprocessing

PROBLEM STATEMENT:

Handle multilingual text input and normalization in Python

<> PYTHON 3 SOURCE CODE

```
import unicodedata

text = "Café STRAßE"
# Unicode normalization and casefolding
normalized = unicodedata.normalize('NFC', text).casefold()

print("Original:  ", text)
print("Normalized:", normalized)
```

>_ EXPECTED TERMINAL OUTPUT

bash terminal • python 09_multilingual_text_normalization.py

```
Original:   Café STRAßE
Normalized: café strasse
```

PROGRAM 10 `10_text_preprocessing_pipeline.py` Foundations, Tokenization & Preprocessing

PROBLEM STATEMENT:

Implement text preprocessing: tokenization, stopword removal, stemming, lemmatization

<> PYTHON 3 SOURCE CODE

```python
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer

text = "The birds were flying and singing happily"
tokens = [w.lower() for w in nltk.word_tokenize(text) if w.isalnum()]
stops = set(stopwords.words('english'))
words = [w for w in tokens if w not in stops]

ps, wn = PorterStemmer(), WordNetLemmatizer()
print("Filtered:  ", words)
print("Stemmed:   ", [ps.stem(w) for w in words])
print("Lemmatized:", [wn.lemmatize(w) for w in words])
```

>_ EXPECTED TERMINAL OUTPUT

bash terminal • python 10_text_preprocessing_pipeline.py

```
Filtered:   ['birds', 'flying', 'singing', 'happily']
Stemmed:    ['bird', 'fli', 'sing', 'happili']
Lemmatized: ['bird', 'flying', 'singing', 'happily']
```

MODULE 2

Corpora, Lexical Resources & Morphology

PROGRAM 11 11_load_and_explore_nltk_corpus.py — Corpora, Lexical Resources & Morphology

PROBLEM STATEMENT:

Load and explore a corpus using NLTK

<> PYTHON 3 SOURCE CODE

```python
import nltk
from nltk.corpus import gutenberg

words = gutenberg.words('austen-emma.txt')

print("Total Words: ", len(words))
print("Unique Vocab:", len(set(words)))
print("Top 3 Words: ", nltk.FreqDist(w.lower() for w in words if w.isalpha()).most_common(3))
```

>_ EXPECTED TERMINAL OUTPUT

```
bash terminal • python 11_load_and_explore_nltk_corpus.py

Total Words:  192427
Unique Vocab: 7811
Top 3 Words:  [('to', 5239), ('the', 5201), ('and', 4896)]
```

PROGRAM 12 12_wordnet_synonyms_antonyms_hypernyms.py — Corpora, Lexical Resources & Morphology

PROBLEM STATEMENT:

Access and query WordNet for synonyms, antonyms, and hypernyms

<> PYTHON 3 SOURCE CODE

```python
from nltk.corpus import wordnet as wn

syn = wn.synsets("good", pos=wn.ADJ)[0]
synonyms = [l.name() for l in syn.lemmas()]
antonyms = [l.antonyms()[0].name() for l in syn.lemmas() if l.antonyms()]
hypernyms = [h.name() for h in syn.hypernyms()]

print("Synonyms: ", synonyms)
print("Antonyms: ", antonyms)
```

>_ EXPECTED TERMINAL OUTPUT

```
bash terminal • python 12_wordnet_synonyms_antonyms_hypernyms.py

Synonyms:  ['good']
Antonyms:  ['bad']
```

PROGRAM 13 `13_parse_xml_linguistic_resource.py` Corpora, Lexical Resources & Morphology

PROBLEM STATEMENT:

Parse and extract information from an XML-based linguistic resource

<> PYTHON 3 SOURCE CODE

```python
import xml.etree.ElementTree as ET

xml_data = '<corpus><word pos="NN">cat</word><word pos="VB">sat</word><word pos="NN">mat</word></corpu
root = ET.fromstring(xml_data)

nouns = [w.text for w in root.findall('word') if w.attrib.get('pos') == 'NN']
verbs = [w.text for w in root.findall('word') if w.attrib.get('pos') == 'VB']

print("Extracted Nouns:", nouns)
print("Extracted Verbs:", verbs)
```

>_ EXPECTED TERMINAL OUTPUT

```
bash terminal • python 13_parse_xml_linguistic_resource.py

Extracted Nouns: ['cat', 'mat']
Extracted Verbs: ['sat']
```

PROGRAM 14 `14_regex_extract_emails_dates_urls.py` Corpora, Lexical Resources & Morphology

PROBLEM STATEMENT:

Write regex patterns to extract emails, dates, and URLs from text

<> PYTHON 3 SOURCE CODE

```python
import re

text = "Contact info@college.edu by 28/10/2026 or check https://college.edu"

emails = re.findall(r'[\w\.-]+@[\w\.-]+', text)
dates = re.findall(r'\d{1,2}/\d{1,2}/\d{2,4}', text)
urls = re.findall(r'https?://\S+', text)

print("Emails:", emails)
print("Dates: ", dates)
print("URLs:  ", urls)
```

>_ EXPECTED TERMINAL OUTPUT

```
bash terminal • python 14_regex_extract_emails_dates_urls.py

Emails: ['info@college.edu']
Dates:  ['28/10/2026']
URLs:   ['https://college.edu']
```

PROGRAM 15 `15_finite_state_automaton_word_recognition.py` Corpora, Lexical Resources & Morphology

PROBLEM STATEMENT:

Implement a simple Finite State Automaton (FSA) for word recognition

<> PYTHON 3 SOURCE CODE

```python
def fsa_recognize_ing(word):
    # Recognizes words ending with 'ing'
    state = 0
    for ch in word.lower():
        if state == 0: state = 1 if ch == 'i' else 0
        elif state == 1: state = 2 if ch == 'n' else (1 if ch == 'i' else 0)
        elif state == 2: state = 3 if ch == 'g' else (1 if ch == 'i' else 0)
        elif state == 3: state = 1 if ch == 'i' else 0
    return state == 3

for w in ["sing", "playing", "walk"]:
    print(f"'{w}': {fsa_recognize_ing(w)}")
```

>_ EXPECTED TERMINAL OUTPUT

```
bash terminal • python 15_finite_state_automaton_word_recognition.py

'sing': True
'playing': True
'walk': False
```

PROGRAM 16 `16_morphological_analyzer_affix_stripping.py` Corpora, Lexical Resources & Morphology

PROBLEM STATEMENT:

Implement a simple morphological analyzer (prefix, suffix stripping)

<> PYTHON 3 SOURCE CODE

```python
def morphological_analyze(word):
    prefixes = ['un', 're', 'dis']
    suffixes = ['ness', 'ing', 'ed', 'ly']
    stem = word.lower()

    prefix = next((p for p in prefixes if stem.startswith(p)), "")
    if prefix: stem = stem[len(prefix):]

    suffix = next((s for s in suffixes if stem.endswith(s)), "")
    if suffix: stem = stem[:-len(suffix)]

    return prefix, stem, suffix

print("unhappiness ->", morphological_analyze("unhappiness"))
print("replaying   ->", morphological_analyze("replaying"))
```

>_ EXPECTED TERMINAL OUTPUT

```
bash terminal • python 16_morphological_analyzer_affix_stripping.py

unhappiness -> ('un', 'happi', 'ness')
replaying   -> ('re', 'play', 'ing')
```

MODULE 3

Sequence Models, POS Tagging & Classification

PROGRAM 17 17_hidden_markov_model_sequence_prediction.py — Sequence Models, POS Tagging & Classification

PROBLEM STATEMENT:

Implement a simple Hidden Markov Model (HMM) for sequence prediction

<> PYTHON 3 SOURCE CODE

```
states = ['Sunny', 'Rainy']
start_p = {'Sunny': 0.6, 'Rainy': 0.4}
trans_p = {'Sunny': {'Sunny': 0.7, 'Rainy': 0.3}, 'Rainy': {'Sunny': 0.4, 'Rainy': 0.6}}
emit_p  = {'Sunny': {'walk': 0.6, 'shop': 0.4}, 'Rainy': {'walk': 0.1, 'shop': 0.9}}

obs = ['walk', 'shop']
# Viterbi decoding algorithm
v0 = {s: start_p[s] * emit_p[s][obs[0]] for s in states}
v1 = {s: max(v0[prev] * trans_p[prev][s] * emit_p[s][obs[1]] for prev in states) for s in states}
best_state = max(v1, key=v1.get)

print("Observations:      ", obs)
print("Likely Final State:", best_state)
```

>_ EXPECTED TERMINAL OUTPUT

bash terminal • python 17_hidden_markov_model_sequence_prediction.py

```
Observations:       ['walk', 'shop']
Likely Final State: Sunny
```

PROGRAM 18 `18_naive_bayes_svm_text_classification.py` Sequence Models, POS Tagging & Classification

PROBLEM STATEMENT:

Train a classifier using Naïve Bayes / SVM for text classification

<> PYTHON 3 SOURCE CODE

```python
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import SVC

X = ["win cash prize", "football match score", "free lottery winner", "player scored goal"]
y = ["Spam", "Sports", "Spam", "Sports"]

vec = CountVectorizer()
X_vec = vec.fit_transform(X)

nb = MultinomialNB().fit(X_vec, y)
svm = SVC().fit(X_vec, y)

test = vec.transform(["cash prize winner", "match goal"])
print("Naive Bayes:", nb.predict(test))
print("SVM:        ", svm.predict(test))
```

>_ EXPECTED TERMINAL OUTPUT

```
bash terminal • python 18_naive_bayes_svm_text_classification.py

Naive Bayes: ['Spam' 'Sports']
SVM:         ['Spam' 'Sports']
```

PROGRAM 19 `19_crf_sequence_labeling_ner.py` Sequence Models, POS Tagging & Classification

PROBLEM STATEMENT:

Apply Conditional Random Fields (CRF) for sequence labeling (NER)

<> PYTHON 3 SOURCE CODE

```python
import sklearn_crfsuite

def word2features(sent):
    return [{'word': w, 'is_title': w.istitle()} for w in sent]

X_train = [word2features(['John', 'lives', 'in', 'London'])]
y_train = [['B-PER', 'O', 'O', 'B-LOC']]

crf = sklearn_crfsuite.CRF()
crf.fit(X_train, y_train)

X_test = [word2features(['Mary', 'visited', 'Paris'])]
print("Input: ", ['Mary', 'visited', 'Paris'])
print("Tagged:", crf.predict(X_test)[0])
```

>_ EXPECTED TERMINAL OUTPUT

bash terminal • python 19_crf_sequence_labeling_ner.py

```
Input:  ['Mary', 'visited', 'Paris']
Tagged: ['B-PER', 'O', 'B-LOC']
```

PROGRAM 20 `20_pos_tagging_nltk.py` Sequence Models, POS Tagging & Classification

PROBLEM STATEMENT:

Perform POS tagging using NLTK

<> PYTHON 3 SOURCE CODE

```python
import nltk

sent = "The quick brown fox jumps over the lazy dog"
tokens = nltk.word_tokenize(sent)
tags = nltk.pos_tag(tokens)

print("Tokens:", tokens[:4])
print("Tags:  ", tags[:4])
```

>_ EXPECTED TERMINAL OUTPUT

bash terminal • python 20_pos_tagging_nltk.py

```
Tokens: ['The', 'quick', 'brown', 'fox']
Tags:   [('The', 'DT'), ('quick', 'JJ'), ('brown', 'NN'), ('fox', 'NN')]
```

PROGRAM 21 `21_stochastic_pos_tagger_hmm.py` Sequence Models, POS Tagging & Classification

PROBLEM STATEMENT:

Implement a stochastic POS tagger using HMM

<> PYTHON 3 SOURCE CODE

```python
import nltk
from nltk.tag import HiddenMarkovModelTagger

train_data = [[('the', 'DT'), ('cat', 'NN'), ('sat', 'VB')],
              [('a', 'DT'), ('dog', 'NN'), ('barked', 'VB')]]

tagger = HiddenMarkovModelTagger.train(train_data)
test_sent = "the dog sat".split()
print("Tagged:", tagger.tag(test_sent))
```

>_ EXPECTED TERMINAL OUTPUT

```
bash terminal • python 21_stochastic_pos_tagger_hmm.py
Tagged: [('the', 'DT'), ('dog', 'NN'), ('sat', 'VB')]
```

PROGRAM 22 `22_transformation_based_learning_tbl.py` Sequence Models, POS Tagging & Classification

PROBLEM STATEMENT:

Demonstrate Transformation-Based Learning (TBL) tagging

<> PYTHON 3 SOURCE CODE

```python
sent = "they will fish and opened the can".split()
# Step 1: Initial baseline tagger (all Nouns)
tags = ['NN'] * len(sent)

# Step 2: Contextual transformation rules
for i in range(len(sent)):
    if i > 0 and sent[i-1] in ['will', 'can']: tags[i] = 'VB'  # Modal -> Verb
    elif sent[i].endswith('ed'): tags[i] = 'VBD'              # Suffix -ed -> Past Verb
    elif i > 0 and sent[i-1] == 'the': tags[i] = 'NN'         # Determiner -> Noun

print("Input: ", sent)
print("Tagged:", list(zip(sent, tags)))
```

>_ EXPECTED TERMINAL OUTPUT

```
bash terminal • python 22_transformation_based_learning_tbl.py
Input:  ['they', 'will', 'fish', 'and', 'opened', 'the', 'can']
Tagged: [('they', 'NN'), ('will', 'NN'), ('fish', 'VB'), ('and', 'NN'), ('opened', 'VBD'), ('the', 'NN'),
('can', 'NN')]
```

PROGRAM 23 `23_handle_unknown_words_pos_tagging.py` Sequence Models, POS Tagging & Classification

PROBLEM STATEMENT:

Handle unknown words in POS tagging

<> PYTHON 3 SOURCE CODE

```python
lexicon = {'the': 'DT', 'is': 'VBZ', 'fast': 'JJ'}

def tag_unknown(w):
    if w.lower() in lexicon: return lexicon[w.lower()]
    if w.isdigit(): return 'CD'        # Number
    if w.endswith('ing'): return 'VBG' # Gerund
    if w.endswith('ed'): return 'VBD'  # Past verb
    if w.endswith('ly'): return 'RB'   # Adverb
    return 'NN'                        # Default open-class noun

sent = "the 2026 player is running quickly".split()
print("Tagged:", [(w, tag_unknown(w)) for w in sent])
```

>_ EXPECTED TERMINAL OUTPUT

```
bash terminal • python 23_handle_unknown_words_pos_tagging.py

Tagged: [('the', 'DT'), ('2026', 'CD'), ('player', 'NN'), ('is', 'VBZ'), ('running', 'VBG'), ('quickly', 'RB')]
```

PROGRAM 24 `24_named_entities_and_multiword_expressions.py` Sequence Models, POS Tagging & Classification

PROBLEM STATEMENT:

Identify named entities and multiword expressions

<> PYTHON 3 SOURCE CODE

```python
import nltk
from nltk.tokenize import MWETokenizer

sent = "Sundar Pichai works at Google in New York"
tokens = nltk.word_tokenize(sent)

# Named Entities
chunks = nltk.ne_chunk(nltk.pos_tag(tokens))
entities = [f"{' '.join(c[0] for c in sub)} ({sub.label()})" for sub in chunks if hasattr(sub, 'label'

# Multiword Expressions
mwe = MWETokenizer([('New', 'York')])
print("Named Entities:       ", entities)
print("Tokenized with MWE:   ", mwe.tokenize(tokens))
```

>_ EXPECTED TERMINAL OUTPUT

```
bash terminal • python 24_named_entities_and_multiword_expressions.py

Named Entities:       ['Sundar (PERSON)', 'Pichai (PERSON)', 'Google (ORGANIZATION)', 'New York (GPE)']
Tokenized with MWE:   ['Sundar', 'Pichai', 'works', 'at', 'Google', 'in', 'New_York']
```

MODULE 4

Context-Free Grammars, Parsing & Trees

PROGRAM 25 `25_cfg_define_and_parse_sentences.py` Context-Free Grammars, Parsing & Trees

PROBLEM STATEMENT:

Define a Context-Free Grammar (CFG) in Python and parse sentences

<> PYTHON 3 SOURCE CODE

```python
import nltk

grammar = nltk.CFG.fromstring("""
S -> NP VP
NP -> Det N
VP -> V NP
Det -> 'the' | 'a'
N -> 'dog' | 'cat'
V -> 'chased'
""")

parser = nltk.ChartParser(grammar)
sent = "the dog chased a cat".split()

for tree in parser.parse(sent):
    print("Parsed Tree:", tree)
```

>_ EXPECTED TERMINAL OUTPUT

bash terminal • python 25_cfg_define_and_parse_sentences.py

```
Parsed Tree: (S (NP (Det the) (N dog)) (VP (V chased) (NP (Det a) (N cat))))
```

PROGRAM 26 `26_top_down_and_bottom_up_parsing.py` Context-Free Grammars, Parsing & Trees

PROBLEM STATEMENT:

Implement top-down and bottom-up parsing

<> PYTHON 3 SOURCE CODE

```
import nltk

grammar = nltk.CFG.fromstring("""
S -> NP VP
NP -> 'John' | 'Mary'
VP -> V NP
V -> 'saw'
""")

sent = "John saw Mary".split()
td = nltk.RecursiveDescentParser(grammar)  # Top-Down Parser
bu = nltk.ShiftReduceParser(grammar)       # Bottom-Up Parser

print("Top-Down Tree: ", list(td.parse(sent))[0])
print("Bottom-Up Tree:", list(bu.parse(sent))[0])
```

>_ EXPECTED TERMINAL OUTPUT

bash terminal • python 26_top_down_and_bottom_up_parsing.py

```
Top-Down Tree:  (S (NP John) (VP (V saw) (NP Mary)))
Bottom-Up Tree: (S (NP John) (VP (V saw) (NP Mary)))
```

PROGRAM 27 `27_chart_parsing_ambiguous_sentences.py` Context-Free Grammars, Parsing & Trees

PROBLEM STATEMENT:

Use chart parsing to analyze ambiguous sentences

<> PYTHON 3 SOURCE CODE

```python
import nltk

grammar = nltk.CFG.fromstring("""
S -> NP VP
NP -> 'I' | Det N | NP PP
VP -> V NP | VP PP
PP -> P NP
Det -> 'the' | 'a'
N -> 'man' | 'telescope'
V -> 'saw'
P -> 'with'
""")

parser = nltk.ChartParser(grammar)
sent = "I saw the man with a telescope".split()
trees = list(parser.parse(sent))

print(f"Ambiguous Parses Count: {len(trees)}")
for i, t in enumerate(trees, 1):
    print(f"Parse Tree {i}:", t)
```

>_ EXPECTED TERMINAL OUTPUT

bash terminal • python 27_chart_parsing_ambiguous_sentences.py

```
Ambiguous Parses Count: 2
Parse Tree 1: (S (NP I) (VP (VP (V saw) (NP (Det the) (N man))) (PP (P with) (NP (Det a) (N telescop
e)))))
Parse Tree 2: (S (NP I) (VP (V saw) (NP (NP (Det the) (N man)) (PP (P with) (NP (Det a) (N telescop
e))))))
```

PROGRAM 28 `28_probabilistic_parsing_pcfg.py` Context-Free Grammars, Parsing & Trees

PROBLEM STATEMENT:

Implement probabilistic parsing using PCFG

<> PYTHON 3 SOURCE CODE

```python
import nltk

grammar = nltk.PCFG.fromstring("""
S -> NP VP [1.0]
NP -> Det N [0.6] | 'John' [0.4]
VP -> V NP [1.0]
Det -> 'the' [1.0]
N -> 'dog' [1.0]
V -> 'saw' [1.0]
""")

parser = nltk.ViterbiParser(grammar)
sent = "John saw the dog".split()

for tree in parser.parse(sent):
    print("Viterbi PCFG Tree:", tree)
    print("Probability:      ", round(tree.prob(), 4))
```

>_ EXPECTED TERMINAL OUTPUT

bash terminal • python 28_probabilistic_parsing_pcfg.py

```
Viterbi PCFG Tree: (S (NP John) (VP (V saw) (NP (Det the) (N dog)))) (p=0.24)
Probability:       0.24
```

PROGRAM 29 `29_load_and_analyze_treebank.py` Context-Free Grammars, Parsing & Trees

PROBLEM STATEMENT:

Load and analyze TreeBank data using NLTK

<> PYTHON 3 SOURCE CODE

```python
import nltk
from nltk.corpus import treebank

tree = treebank.parsed_sents()[0]

print("Treebank Sentence Words:", tree.leaves()[:5])
print("Sample Subtree (NP):    ", [s for s in tree.subtrees() if s.label() == 'NP'][0])
```

>_ EXPECTED TERMINAL OUTPUT

bash terminal • python 29_load_and_analyze_treebank.py

```
Treebank Sentence Words: ['Pierre', 'Vinken', ',', '61', 'years']
Sample Subtree (NP):     (NP (NNP Pierre) (NNP Vinken))
```

PROGRAM 30 `30_evaluate_parsing_precision_recall.py` Context-Free Grammars, Parsing & Trees

PROBLEM STATEMENT:

Evaluate parsing performance using precision and recall

<> PYTHON 3 SOURCE CODE

```python
# Extracted labeled constituent brackets: (Label, start_idx, end_idx)
gold = {('NP', 0, 2), ('VP', 2, 4), ('S', 0, 4)}
cand = {('NP', 0, 2), ('VP', 2, 3), ('S', 0, 4)}

matched = len(gold & cand)
precision = matched / len(cand)
recall = matched / len(gold)
f1 = 2 * precision * recall / (precision + recall)

print(f"Precision: {precision:.2f}, Recall: {recall:.2f}, F1: {f1:.2f}")
```

>_ EXPECTED TERMINAL OUTPUT

bash terminal • python 30_evaluate_parsing_precision_recall.py

```
Precision: 0.67, Recall: 0.67, F1: 0.67
```

PROGRAM 31 `31_visualize_parse_trees.py` Context-Free Grammars, Parsing & Trees

PROBLEM STATEMENT:

Visualize parse trees for given sentences

<> PYTHON 3 SOURCE CODE

```python
from nltk.tree import Tree

tree = Tree.fromstring("(S (NP (Det The) (N cat)) (VP (V sat)))")

print("Bracket Representation:")
print(tree)
print("\nASCII Visualization:")
tree.pretty_print()
```

>_ EXPECTED TERMINAL OUTPUT

bash terminal • python 31_visualize_parse_trees.py

```
Bracket Representation:
(S (NP (Det The) (N cat)) (VP (V sat)))

ASCII Visualization:
        S
    ____|____
   NP        VP
  __|___     |
Det    N     V
 |     |     |
The   cat   sat
```

MODULE 5

Semantics, WordNet & Word Sense Disambiguation

PROGRAM 32 `32_sentence_meaning_logical_forms.py` Semantics, WordNet & Word Sense Disambiguation

PROBLEM STATEMENT:

Represent sentence meaning using logical forms

<> PYTHON 3 SOURCE CODE

```python
from nltk.sem.logic import Expression

expr1 = Expression.fromstring("Loves(john, mary)")
expr2 = Expression.fromstring("all x.(Student(x) -> Studies(x))")

print("Sentence 1 FOL:", expr1)
print("Sentence 2 FOL:", expr2)
```

>_ EXPECTED TERMINAL OUTPUT

bash terminal • python 32_sentence_meaning_logical_forms.py

```
Sentence 1 FOL: Loves(john,mary)
Sentence 2 FOL: all x.(Student(x) -> Studies(x))
```

PROGRAM 33 `33_lexical_semantics_wordnet_similarity.py` Semantics, WordNet & Word Sense Disambiguation

PROBLEM STATEMENT:

Perform lexical semantic analysis using WordNet similarity measures

<> PYTHON 3 SOURCE CODE

```python
from nltk.corpus import wordnet as wn

c1 = wn.synsets("cat", pos=wn.NOUN)[0]
c2 = wn.synsets("dog", pos=wn.NOUN)[0]

print("Path Similarity:            ", round(c1.path_similarity(c2), 4))
print("Leacock-Chodorow Similarity:", round(c1.lch_similarity(c2), 4))
print("Wu-Palmer Similarity:       ", round(c1.wup_similarity(c2), 4))
```

>_ EXPECTED TERMINAL OUTPUT

bash terminal • python 33_lexical_semantics_wordnet_similarity.py

```
Path Similarity:            0.2
Leacock-Chodorow Similarity: 2.0281
Wu-Palmer Similarity:       0.8571
```

PROGRAM 34 `34_semantic_network_dataset.py` Semantics, WordNet & Word Sense Disambiguation

PROBLEM STATEMENT:

Build a semantic network from a small dataset

<> PYTHON 3 SOURCE CODE

```python
import networkx as nx

G = nx.DiGraph([
    ("Canary", "Bird", {'rel': 'is-a'}),
    ("Bird", "Animal", {'rel': 'is-a'}),
    ("Canary", "Yellow", {'rel': 'color'}),
    ("Bird", "Wings", {'rel': 'has'})
])

print("Nodes:", list(G.nodes()))
print("Edges:", [(u, v, d['rel']) for u, v, d in G.edges(data=True)])
```

>_ EXPECTED TERMINAL OUTPUT

```
bash terminal • python 34_semantic_network_dataset.py
Nodes: ['Canary', 'Bird', 'Animal', 'Yellow', 'Wings']
Edges: [('Canary', 'Bird', 'is-a'), ('Canary', 'Yellow', 'color'), ('Bird', 'Animal', 'is-a'), ('Bird', 'Wings', 'has')]
```

PROGRAM 35 `35_lesk_algorithm_wsd.py` Semantics, WordNet & Word Sense Disambiguation

PROBLEM STATEMENT:

Implement the Lesk algorithm for WSD

<> PYTHON 3 SOURCE CODE

```python
import nltk
from nltk.wsd import lesk

sent1 = "I deposited money in the bank".split()
sent2 = "The fisherman sat on the river bank".split()

sense1 = lesk(sent1, "bank", pos='n')
sense2 = lesk(sent2, "bank", pos='n')

print("Sense 1 (Finance):", sense1.name(), "-", sense1.definition())
print("Sense 2 (River):  ", sense2.name(), "-", sense2.definition())
```

>_ EXPECTED TERMINAL OUTPUT

```
bash terminal • python 35_lesk_algorithm_wsd.py
Sense 1 (Finance): savings_bank.n.02 - a container (usually with a slot in the top) for keeping money at home
Sense 2 (River):   bank.n.01 - sloping land (especially the slope beside a body of water)
```

PROGRAM 36 `36_wordnet_disambiguate_meanings_in_context.py` Semantics, WordNet & Word Sense Disambiguation

PROBLEM STATEMENT:

Use WordNet to disambiguate word meanings in context

<> PYTHON 3 SOURCE CODE

```python
from nltk.corpus import wordnet as wn

sent = "The cat caught a mouse"
context_syn = wn.synsets("cat", pos=wn.NOUN)[0]

# Disambiguate 'mouse' by selecting synset with highest similarity to context word 'cat'
mouse_senses = wn.synsets("mouse", pos=wn.NOUN)
best_sense = max(mouse_senses, key=lambda s: context_syn.wup_similarity(s) or 0)

print("Target Word: 'mouse' in context 'The cat caught a mouse'")
print("Disambiguated Sense:", best_sense.name(), "-", best_sense.definition())
```

>_ EXPECTED TERMINAL OUTPUT

```
bash terminal • python 36_wordnet_disambiguate_meanings_in_context.py

Target Word: 'mouse' in context 'The cat caught a mouse'
Disambiguated Sense: mouse.n.01 - any of numerous small rodents...
```

PROGRAM 37 `37_supervised_wsd_machine_learning.py` Semantics, WordNet & Word Sense Disambiguation

PROBLEM STATEMENT:

Build a supervised WSD model using machine learning

<> PYTHON 3 SOURCE CODE

```python
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

# Polysemous word 'bass': Fish vs Music
X = ["catch sea bass fish", "eat fresh sea bass", "play electric bass guitar", "loud bass music notes"]
y = ["Fish", "Fish", "Music", "Music"]

vec = CountVectorizer()
clf = MultinomialNB().fit(vec.fit_transform(X), y)

test = ["fresh sea bass", "loud bass music"]
print("Predictions:", list(zip(test, [str(p) for p in clf.predict(vec.transform(test))])))
```

>_ EXPECTED TERMINAL OUTPUT

```
bash terminal • python 37_supervised_wsd_machine_learning.py

Predictions: [('fresh sea bass', 'Fish'), ('loud bass music', 'Music')]
```

MODULE 6

Text Coherence, Translation, IR & Summarization

PROGRAM 38 38_text_coherence_sentence_similarity.py — Text Coherence, Translation, IR & Summarization

PROBLEM STATEMENT:

Analyze text coherence using sentence similarity

<> PYTHON 3 SOURCE CODE

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

sents = [
    "Python is great for NLP programming.",
    "NLP allows computers to understand text.",
    "Computers use text to learn language."
]

vecs = TfidfVectorizer().fit_transform(sents)
# Measure similarity between consecutive adjacent sentences
coherence = [float(cosine_similarity(vecs[i], vecs[i+1])[0][0]) for i in range(len(sents)-1)]

print("Pairwise Similarities: ", [round(s, 2) for s in coherence])
print("Overall Coherence Score:", round(sum(coherence) / len(coherence), 2))
```

>_ EXPECTED TERMINAL OUTPUT

```
bash terminal • python 38_text_coherence_sentence_similarity.py

Pairwise Similarities:  [0.12, 0.38]
Overall Coherence Score: 0.25
```

PROGRAM 39 `39_simple_machine_translation_sequence_model.py` Text Coherence, Translation, IR & Summarization

PROBLEM STATEMENT:

Build a simple Machine Translation system using sequence models

PYTHON 3 SOURCE CODE

```python
# Sequence-to-sequence translation of words and phrases (English -> French)

seq_model = {
    "hello": "bonjour",
    "how are you": "comment vas-tu",
    "thank you": "merci",
    "good morning": "bonjour"
}

def translate_sequence(sentence):
    text = sentence.lower().strip()
    if text in seq_model:
        return seq_model[text]
    word_map = {"how": "comment", "are": "vas", "you": "tu", "thank": "merci"}
    return " ".join(word_map.get(w, w) for w in text.split())

for s in ["Hello", "How are you", "Thank you"]:
    print(f"'{s}' -> '{translate_sequence(s)}'")
```

EXPECTED TERMINAL OUTPUT

```
bash terminal • python 39_simple_machine_translation_sequence_model.py

'Hello' -> 'bonjour'
'How are you' -> 'comment vas-tu'
'Thank you' -> 'merci'
```

PROGRAM 40 `40_information_retrieval_tfidf.py` Text Coherence, Translation, IR & Summarization

PROBLEM STATEMENT:

Implement Information Retrieval using TF-IDF

<> PYTHON 3 SOURCE CODE

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

docs = [
    "Python is for data science and AI",
    "Deep learning models use neural networks",
    "SQL databases store structured relational records"
]

vec = TfidfVectorizer()
X = vec.fit_transform(docs)

query = ["neural networks deep learning"]
sims = cosine_similarity(vec.transform(query), X)[0]
best_idx = sims.argmax()

print("Query:     ", query[0])
print(f"Best Match (Doc {best_idx + 1}, score={sims[best_idx]:.2f}): {docs[best_idx]}")
```

>_ EXPECTED TERMINAL OUTPUT

bash terminal • python 40_information_retrieval_tfidf.py

```
Query:      neural networks deep learning
Best Match (Doc 2, score=0.74): Deep learning models use neural networks
```

PROGRAM 41 `41_text_classification_spam_sentiment.py` Text Coherence, Translation, IR & Summarization

PROBLEM STATEMENT:

Perform text classification (spam detection / sentiment analysis)

<> PYTHON 3 SOURCE CODE

```python
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

texts = ["win cash prize claim now", "important meeting notes", "win free prize lottery", "call me tom
labels = ["Spam", "Ham", "Spam", "Ham"]

vec = CountVectorizer()
clf = MultinomialNB().fit(vec.fit_transform(texts), labels)

test = ["free prize winner", "see you at meeting"]
print("Input:      ", test)
print("Predictions:", [str(p) for p in clf.predict(vec.transform(test))])
```

>_ EXPECTED TERMINAL OUTPUT

```
bash terminal • python 41_text_classification_spam_sentiment.py

Input:       ['free prize winner', 'see you at meeting']
Predictions: ['Spam', 'Ham']
```

PROGRAM 42 `42_extractive_text_summarization.py` Text Coherence, Translation, IR & Summarization

PROBLEM STATEMENT:

Generate text summaries using extractive methods

<> PYTHON 3 SOURCE CODE

```python
from collections import Counter
from nltk.tokenize import sent_tokenize, word_tokenize

text = "NLP is a branch of AI. It helps computers understand human language. NLP is useful for transla
sents = sent_tokenize(text)
words = [w.lower() for w in word_tokenize(text) if w.isalnum()]
freq = Counter(words)

# Score sentences by word frequency
scores = {s: sum(freq[w.lower()] for w in word_tokenize(s) if w.isalnum()) for s in sents}
top_sent = max(scores, key=scores.get)

print("Original Sentences:", len(sents))
print("Extractive Summary:", top_sent)
```

>_ EXPECTED TERMINAL OUTPUT

```
bash terminal • python 42_extractive_text_summarization.py

Original Sentences: 3
Extractive Summary: NLP is useful for translation and chatbots.
```